

AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

ALU mechanics, shifted-register operands, bitwise operations, condition flags, structured branches, and conditional selection

Dr. J. Burns

Topic 4 Outline

1. ALU Operations: Arithmetic, Shifting, & Bitwise Logic

- Addition, subtraction, multiplication, division, and shifted-register operands.
- Bitwise logic with `and`, `orr`, `eor`, and `bic`.

2. Condition Flags & Comparisons

- NZCV flags, `cmp`, and the signed vs. unsigned branch distinction.

3. Control Flow: Branches & Selection

- Translating `if-else` structures and loops.
- Register zero tests with `cbz/cbnz` and conditional selection with `csel`.

PART 1

ALU Operations: Arithmetic, Shifting, & Bitwise Logic

Arithmetic Primitives: Addition & Subtraction

- Arithmetic instructions compute integer results in registers.
- Some arithmetic instructions can also update **condition flags**: processor-state bits that record properties of the result for later comparisons and branches.

Instruction	Operation	Sets condition flags?
add x0, x1, x2	$x_0 \leftarrow x_1 + x_2$	No
adds x0, x1, x2	$x_0 \leftarrow x_1 + x_2$	Yes
sub x0, x1, x2	$x_0 \leftarrow x_1 - x_2$	No
subs x0, x1, x2	$x_0 \leftarrow x_1 - x_2$	Yes
neg x0, x1	$x_0 \leftarrow 0 - x_1$	No; alias for sub x0, xzr, x1

Shifted-Register Operands and Shift Operations

- Many AArch64 arithmetic instructions can apply a shift to one register operand as part of the operation.
- This is useful for simple scaling, such as multiplying an index by a power of two.

Shift Type	Mnemonic	Operation Meaning
Logical Shift Left	lsl #k	Multiply by 2^k modulo the register width.
Logical Shift Right	lsr #k	Unsigned division by 2^k ; zeros enter the high bits.
Arithmetic Shift Right	asr #k	Signed right shift; the sign bit is copied into the high bits.

Integrated Shift-Add Examples

```
add x0, x1, x2, lsl #3     $x_0 \leftarrow x_1 + (x_2 \times 8)$ 
sub x0, x1, x2, lsr #1     $x_0 \leftarrow x_1 - \lfloor x_2/2 \rfloor$  for unsigned  $x_2$ 
```

Integer Multiplication & Division

- AArch64 provides separate integer instructions for multiplication and division.
- Division instructions return an integer quotient; any remainder is discarded.

Instruction	Purpose
<code>mul x0, x1, x2</code>	Multiply two registers; keep the low 64 bits.
<code>sdiv x0, x1, x2</code>	Signed integer division.
<code>udiv x0, x1, x2</code>	Unsigned integer division.

Choose `sdiv` or `udiv` based on how the operands should be interpreted: signed values use `sdiv`; unsigned counts, sizes, and addresses usually use `udiv`.

Bitwise Logic Primitives

- Bitwise operations apply independently to each corresponding bit position.

Instruction	Operation	Bitwise Formula
<code>and x0, x1, x2</code>	Bitwise AND	$x_0 \leftarrow x_1 \wedge x_2$
<code>orr x0, x1, x2</code>	Bitwise OR	$x_0 \leftarrow x_1 \vee x_2$
<code>eor x0, x1, x2</code>	Bitwise XOR	$x_0 \leftarrow x_1 \oplus x_2$
<code>bic x0, x1, x2</code>	Bit Clear	$x_0 \leftarrow x_1 \wedge (\sim x_2)$

Bit Clear (`bic`) Idiom

`bic x0, x1, x2` clears bits in x_1 wherever the mask in x_2 has 1s. A common use is clearing a flag or disabling an option stored in one bit, without disturbing the other settings packed into the same value.

PART 2

Condition Flags & Comparison Mechanics

NZCV Condition Flags for Control Flow

- Topic 4 is where we use the pstate condition flags in assembly programs.
- The NZCV bits record the status of the most recent instruction that updates them.
- Common flag-setting forms include instructions with an “s” suffix and explicit comparisons.

Flag	Name	Condition Set to 1
N	Negative	The most-significant bit of the result is 1.
Z	Zero	Result is exactly zero.
C	Carry	Unsigned carry-out (addition) or no borrow (subtraction).
V	Overflow	The signed mathematical result cannot be represented in the destination width.

Carry Flag in Subtraction

In AArch64 subtraction ($A - B$), $C = 1$ indicates $A \geq B$ (no borrow required). $C = 0$ indicates $A < B$ (unsigned borrow occurred).

Reading NZCV After Arithmetic

- Read the flags as a short summary of the **last flag-setting operation**.
- For `cmp x0, x1`, treat the flags as describing the subtraction $x0 - x1$; the numeric result is discarded.

Simple Examples

- If $x0 == x1$, then `cmp x0, x1` produces zero, so **Z = 1**.
- If $x0 < x1$ in an unsigned subtraction, a borrow is needed, so **C = 0**.
- If a result has its sign bit set, **N = 1**.
- If a signed result cannot fit, such as 32-bit $0x7fffffff + 1$, **V = 1**.

Comparison with `cmp`

- `cmp` compares two operands by subtracting one from the other and updating the condition flags.
- `cmp x0, x1` is really `subs xzr, x0, x1`: subtract, update NZCV, and discard the numerical result in the zero register.

Instruction	Equivalent Operation	Meaning
<code>cmp x0, x1</code>	<code>subs xzr, x0, x1</code>	Compare x_0 with x_1 .
<code>cmp x0, #0</code>	<code>subs xzr, x0, #0</code>	Test whether x_0 is zero, negative, or positive.

After `cmp x0, x1`, a following conditional branch interprets the flags according to the intended signed or unsigned comparison.

Condition Codes: Signed vs. Unsigned Branches

Branch	Meaning	Flag Condition	Comparison Context
b.eq	Equal	$Z == 1$	Signed / Unsigned
b.ne	Not Equal	$Z == 0$	Signed / Unsigned
b.lt	Signed Less Than ($<$)	$N \neq V$	Signed
b.le	Signed Less or Equal ($\leq$)	$(Z == 1) \vee (N \neq V)$	Signed
b.gt	Signed Greater Than ($>$)	$(Z == 0) \wedge (N == V)$	Signed
b.ge	Signed Greater or Equal ($\geq$)	$N == V$	Signed
b.lo	Unsigned Lower ($<$)	$C == 0$	Unsigned
b.hs	Unsigned Higher or Same ($\geq$)	$C == 1$	Unsigned
b.hi	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	Unsigned
b.ls	Unsigned Lower or Equal ($\leq$)	$(C == 0) \vee (Z == 1)$	Unsigned

Usage Context

Unsigned conditions are appropriate when the program semantics treat the values as unsigned; they are also commonly used for address and range comparisons.

PART 3

Control Flow: Branches & Selection

Compare-and-Branch Instructions: **cbz** and **cbnz**

- **cbz** / **cbnz** test a register directly and do not alter NZCV.
- They combine a zero test and branch in one architectural instruction.

Null Pointer Validation

```
cbz x0, .Lnull    // branch if null
ldr x1, [x0]      // dereference
```

Loop Counter Decrement

```
.Lloop:
sub x1, x1, #1    // count-
cbnz x1, .Lloop   // repeat if nonzero
```

Translating if-then-else Blocks

High-Level C Code

```
if (x > y) {  
    result = x - y;  
} else {  
    result = y - x;  
}
```

AArch64 Assembly Mapping

```
// Assume: x, y are signed int64_t  
// x0=x, x1=y, x2=result  
cmp x0, x1           // Compare x and y  
b.le .Lelse          // Invert test: jump if x <= y  
sub x2, x0, x1        // Then-body: result = x - y  
b .Lend              // Skip else block  
.Lelse:  
sub x2, x1, x0        // Else-body: result = y - x  
.Lend:
```

Assembly Pattern: Invert the high-level condition to fall through into the “then” path.

Loop Translation: Inverted Loop Pattern

High-Level C Loop

```
// Compute array sum
int64_t i = 0;
int64_t n = ...;
while (i < n) {
    sum += arr[i];
    i++;
}
```

AArch64 Loop with Test at the End

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr           // i = 0
mov x3, xzr           // sum = 0
b .Ltest              // test first
.Lbody:
ldr x4, [x0, x2, lsl #3] // load arr[i]
add x3, x3, x4         // sum += arr[i]
add x2, x2, #1         // i++
.Ltest:
cmp x2, x1             // i < n?
b.lt .Lbody            // repeat
```

Common Loop Layout: After the first jump to the test, each repeat runs the body and then checks whether to loop again.

Conditional Selection with `cse`

- `cse` chooses between two register operands using the current condition flags.
- It is useful for short value selections where both candidate values are already available in registers.

Signed Maximum: $\text{result} = (x_1 > x_2) ? x_1 : x_2$

```
cmp x1, x2           // set flags from x1 - x2
cse x0, x1, x2, gt    // if x1 > x2, choose x1; else choose x2
```

The condition code on `cse` is interpreted the same way as a conditional branch condition.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 4: Summary

- **Arithmetic & Shifts:** AArch64 supports register arithmetic, flag-setting variants, and shifted-register operands.
- **Bitwise Operations:** `and`, `orr`, `eor`, and `bic` operate directly on packed binary data.
- **NZCV Flags:** Flag-setting instructions record result properties used by later comparisons and branches.
- **Comparisons:** `cmp` subtracts, updates NZCV, and discards the numerical result.
- **Control Flow:** Conditional branches, `cbz/cbnz`, and `csel` translate common decisions and loops.

Self-Test Questions: Arithmetic & Logic

- ❶ **Shifted Addition:** What is the exact mathematical operation and resulting decimal value stored in `x0` after executing:

```
mov x1, #12  mov x2, #5  add x0, x1, x2, lsl #2
```

- ❷ **Bit Clear Operation:** If register `x1` contains `0xff` and `x2` contains `0x0f`, what value resides in `x0` after executing `bic x0, x1, x2`?
- ❸ **Signed vs. Unsigned Comparisons:** If `x0` contains `-1` (`0xffffffff_ffffffff`) and `x1` contains `1`, which branch will be taken after `cmp x0, x1: b.lt` or `b.lo`? Why?
- ❹ **Flag Updating:** Explain why `add x0, x1, x2` does not update the Zero (*Z*) flag even if the resulting sum is exactly zero.

Self-Test Questions: Control Flow & Selection

- ❶ **Loop Translation:** In the loop example, why does the code jump to the test before the first iteration?
- ❷ **Branchless Translation:** Write an equivalent 3-instruction branchless AArch64 sequence using `cmp`, `neg`, and `csel` to implement:

```
int64_t abs_val = (val < 0) ? -val : val;
```

- ❸ **Compare-and-Branch Instructions:** When should an assembly programmer use `cbz x0, label` instead of `cmp x0, #0` followed by `b.eq label`?
- ❹ **Signed vs. Unsigned Branches:** Why are `b.lt` and `b.lo` not interchangeable after the same `cmp` instruction?

Looking Ahead

Next Lecture Preview

Our Next Topic: Memory Access & AArch64 Addressing Modes

- **Load/Store Discipline:** Moving values between registers and memory.
- **Transfer Widths:** Working with byte, halfword, word, and doubleword data.
- **Effective Addresses:** Forming addresses from base registers, offsets, and indexes.
- **Arrays & Pointers:** Applying addressing modes to common memory-access patterns.